ECE
ÉCOLE D'INGÉNIEURS
ENGINEERING SCHOOL

Cours Base de données

Cours 6
Le Langage SQL (Partie 3)

Jointures, sous requêtes et vues

- Niveau: B1
- Année universitaire: 2025-2026

Dr. Salma Sassi
salma.sassi@ece.fr

Nous avons appris

- Sélectionner des données avec SELECT et WHERE
- Trier et filtrer les résultats (ORDER BY, DISTINCT)
- Regrouper et agréger les données (GROUP BY, HAVING)

Nous allons apprendre à **naviguer entre les tables, créer des sous requêtes & des vues**

Si les données dont nous avons besoin étaient réparties sur plusieurs tables ?

1. Comment peut on relier des informations provenant de différentes tables ?
2. Comment vérifier que les données correspondent correctement ?
3. Quelle méthode utiliser pour combiner ces tables et obtenir les résultats attendus ?

Naviguer entre les tables: Jointure

Requête: *« Afficher la liste des étudiants avec les détails de leur département»*

Table 1: Département

dep_num	dep_nom
01	Ain
75	Paris
77	Seine-et-Marne
92	Hauts-de-Seine
93	Seine-Saint-Denis

Clé primaire

Table 2: Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Salma	Sassi	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

Clé secondaire

nom	prenom	dateN	note	Dep-num	licence
Martin	Véra	85-10-31	13.5	77	2

Pour lier ces données, une jointure est obligatoire

Une jointure, qu'est ce que c'est?

- **Définition**
  - Une jointure relie **plusieurs tables** dans une base de données pour combiner leurs données.
  - Elle permet d'**extraire les informations liées** dans une seule requête.

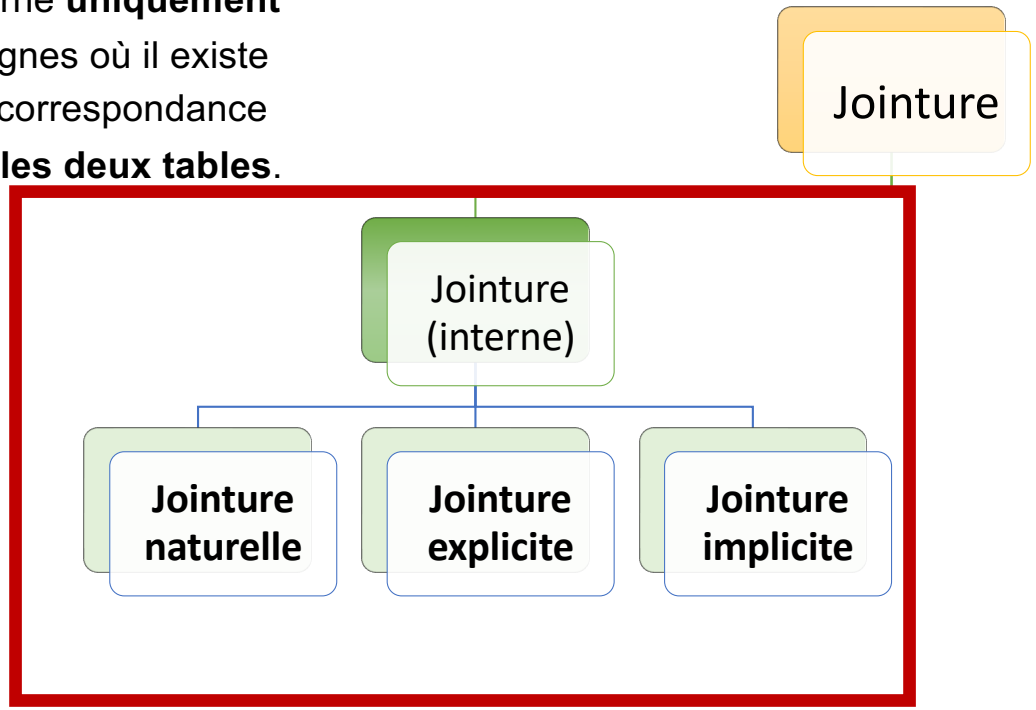
SQL1 vs SQL2 :

- **SQL1 :** Syntaxe simple, mais limitée pour certaines opérations (WHERE).
- **SQL2 :** Syntaxe moderne, plus claire et plus puissante (ex : INNER JOIN, LEFT JOIN, etc.).

Quels sont les types de jointures et comment les utiliser ?

Les types de jointure

Retourne **uniquement** les lignes où il existe une correspondance dans **les deux tables**.



La jointure interne

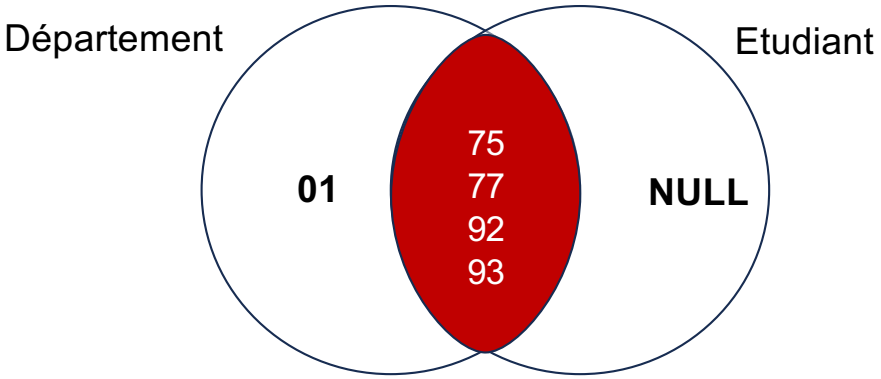
Table 1: Département

dep_num	dep_nom
01	Ain
75	Paris
77	Seine-et-Marne
92	Hauts-de-Seine
93	Seine-Saint-Denis

Table 2: Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	**77**	2
Martin	Annie	85-12-31	15.5	**75**	1
Dupont	Sylvie	83-02-03	15.0	**77**	2
Salma	Sassi	83-10-22	05.7	**93**	1
Dupond	Laurent			**92**	2
Lefèvre	Laurent		11.5		3

Jointure Interne



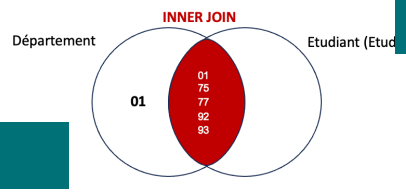
Jointure avec les colonnes **où il existe une correspondance** dans les deux tables.

La jointure interne

	Jointure naturelle (Natural JOIN)	Jointure explicite (INNER JOIN)	Jointure implicite (WHERE)
Définition	Jointure automatique sur les colonnes ayant le même nom.	Jointure avec condition précisée dans ON.	Liste des tables avec relation spécifiée dans WHERE.
Syntaxe	SELECT * FROM table1 NATURAL JOIN table2;	SELECT * FROM table1 **INNER JOIN** table2 **ON** table1.col = table2.col;	SELECT * FROM table1, table2 **WHERE** table1.col = table2.col;
Exemple en SQL « Afficher la liste des étudiants avec les détails de leur département»	SELECT * FROM Etudiant **Natural JOIN** département	SELECT * FROM Etudiant e **INNER JOIN** département d **ON** e.dep=d.dep_num;	SELECT * FROM Etudiant e, département d **WHERE** e.dep=d.dep_num ;

NB: Le mot clé INNER est facultatif !

La jointure interne



Requête: *« Afficher la liste des étudiants avec les détails de leur département»*

```
SELECT * FROM Etudiant e INNER JOIN  Département d ON e.dep=d.dep_num;
SELECT * FROM Etudiant e, Département d  WHERE e.dep=d.dep_num ;
```

Résultat

Etudiant → | Département →

nom	prenom	dateN	note	Dep_num	licence	dep	dep_nom
Martin	Véra	85-10-31	13.5	77	2	77	Seine-et-Marne
Martin	Annie	85-12-31	15.5	75	1	75	Paris
Dupont	Sylvie	83-02-03	15.0	77	2	77	Seine-et-Marne
Salma	Sassi	83-10-22	05.7	93	1	93	Seine-Saint-Denis
Dupond	Laurent			92	2	92	Hauts-de-Seine

Les colonnes utilisées pour la jointure apparaissent deux fois dans le résultat, sauf si spécifiées autrement (dans le Select).

```
SELECT * FROM Etudiant Natural JOIN Département
```

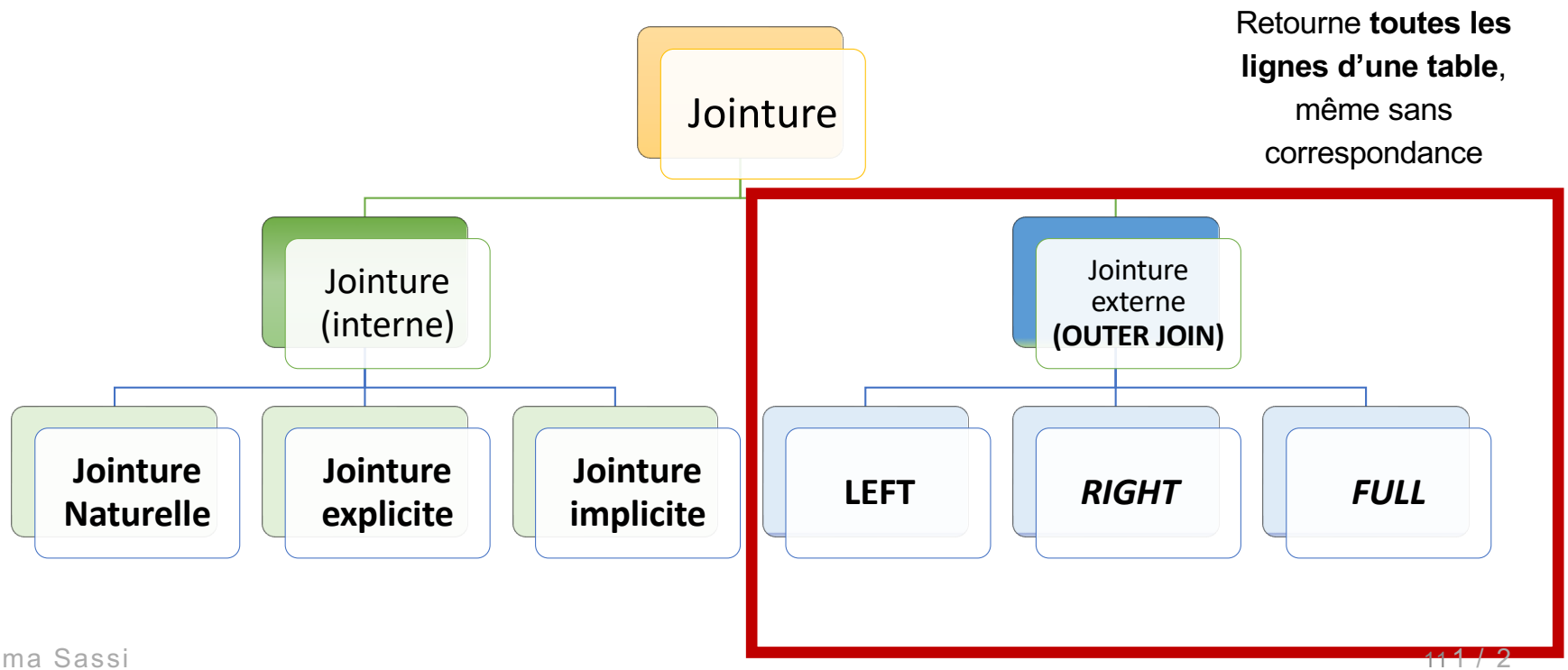
nom	prenom	dateN	note	Dep_num	licence	dep_nom
Martin	Véra	85-10-31	13.5	77	2	Seine-et-Marne
Martin	Annie	85-12-31	15.5	75	1	Paris
Dupont	Sylvie	83-02-03	15.0	77	2	Seine-et-Marne
Salma	Sassi	83-10-22	05.7	93	1	Seine-Saint-Denis
Dupond	Laurent			92	2	Hauts-de-Seine

Pas de doublons

La jointure interne – Comparatif

	Jointure naturelle (Natural JOIN)	Jointure explicite (INNER JOIN)	Jointure implicite (WHERE)
Description	Automatique sur colonnes communes	Explicite avec condition ON	Condition dans WHERE
Syntaxe	NATURAL JOIN	INNER JOIN ... ON ...	FROM table1, table2 WHERE ...
Clarté	Moyenne (Peu d'explicitation)	Elevée (Condition explicite dans ON)	Faible (Relation noyée dans WHERE)
Flexibilité	Faible (dépend des noms communs)	Très flexible (permet des conditions variées)	Moyenne
Doublons	Non	Oui, par défaut	Oui, par défaut
Usage recommandé	Cas simple avec noms identiques	Recommandé pour tous les projets	A éviter si possible

Les types de jointure



Retourne **toutes les lignes d'une table**, même sans correspondance

Comprendre les tables gauche et droite

Table de gauche

Etudiant

Jointure externe

Département

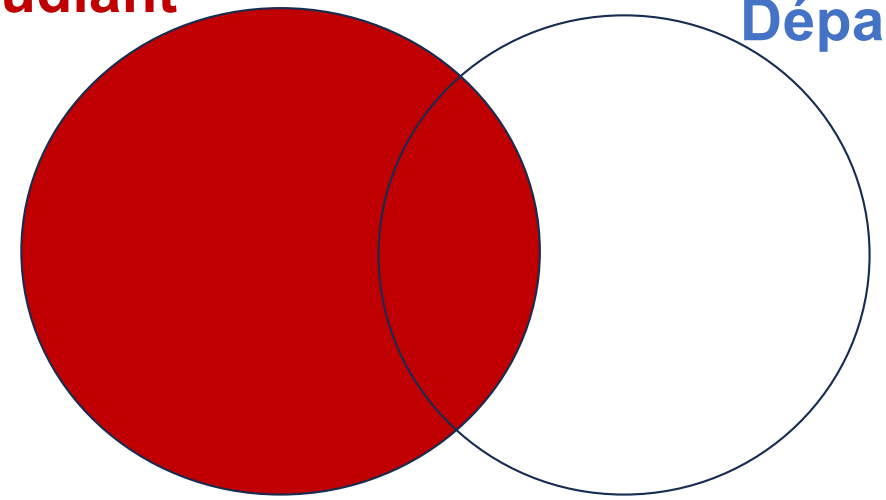


Diagramme de Venn

```
SELECT * FROM Etudiant
LEFT JOIN Département
ON dep=dep_num ;
```

Table de droite

Jointure externe **(LEFT OUTER JOIN)**

Permet de lister en plus des éléments communs à 2 tables tous les éléments de la table de gauche (= Table du FROM)

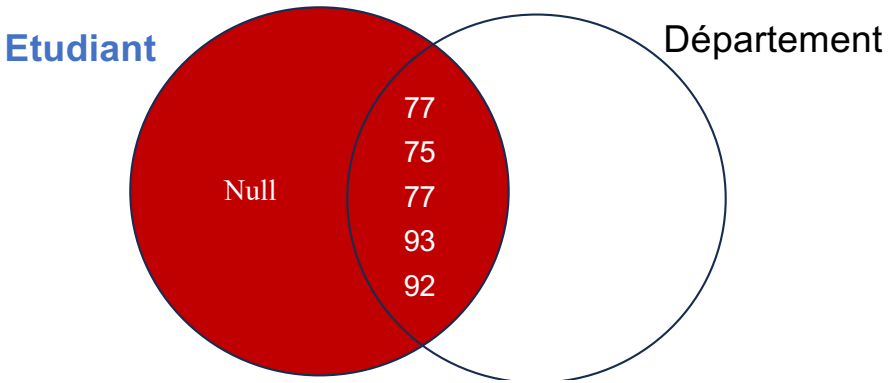
Table 1: Département

dep_num	dep_nom
01	Ain
75	Paris
77	Seine-et-Marne
92	Hauts-de-Seine
93	Seine-Saint-Denis

Table 2: Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Martin	Annie	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

LEFT OUTER JOIN



Retourner tous les enregistrements de la table de gauche même s'il n'y a pas de correspondance avec la table de droite

↓

Inclut toutes les lignes de la table Etudiant (gauche), même celles sans correspondance dans Département (droite), avec des valeurs NULL."

Jointure externe **(LEFT OUTER JOIN)**

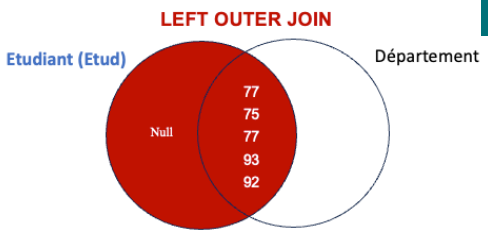


Table de gauche

```
SELECT * FROM Etudiant
LEFT OUTER JOIN Département  ON dep=dep_num ;
```

Etudiant

Département

nom	prenom	dateN	note	dep	licence	dep_num	dep_nom
Martin	Véra	85-10-31	13.5	77	2	77	Seine-et-Marne
Martin	Annie	85-12-31	15.5	75	1	75	Paris
Dupont	Sylvie	83-02-03	15.0	77	2	77	Seine-et-Marne
Salma	Sassi	83-10-22	05.7	93	1	93	Seine-Saint-Denis
Dupond	Laurent	null	null	92	2	92	Hauts-de-Seine
Lefèvre	Laurent	null	11.5	null	3	null	null

Les colonnes dep_num et dep_nom sont remplies avec des NULL lorsqu'il n'y a pas de correspondance avec Département

Jointure externe **(LEFT OUTER JOIN)**

- **Définition**
  - Une **jointure par la gauche**, où la **table principale** (table de gauche) est utilisée comme référence.
  - Toutes les lignes de la **table principale** sont incluses, même si aucune correspondance n'existe dans la table jointe (droite).
  - Les valeurs des colonnes de la table droite seront **NULL** s'il n'y a pas de correspondance.
  - **Alias :** C'est un **LEFT OUTER JOIN** par défaut.

- **Syntaxe**

```
SELECT * FROM main_table
LEFT JOIN joined_table ON main_table.id = joined_table.main_id;
```

- **Cas d'usage**

1. Obtenir la liste des **étudiants** avec ou sans **départements attribués**

Jointure externe **(Right OUTER JOIN)**

Inclut toutes les lignes de la table de droite, même celles sans correspondance dans la table de gauche.

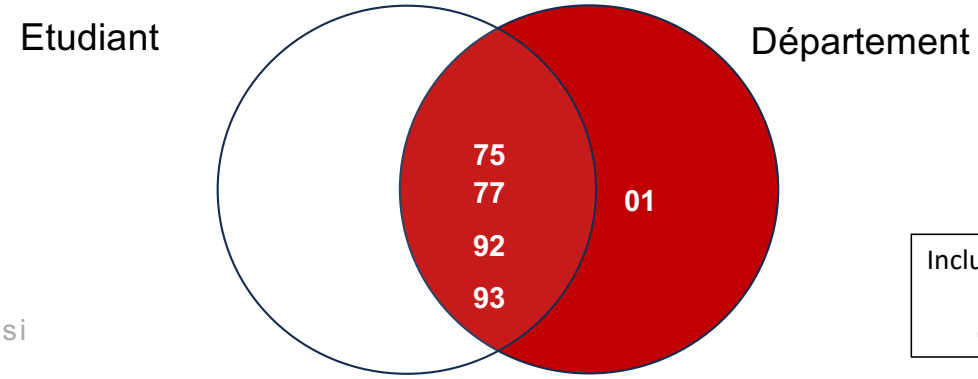
Table 1: Département

dep_num	dep_nom
01	Ain
75	Paris
77	Seine-et-Marne
92	Hauts-de-Seine
93	Seine-Saint-Denis

Table 2: Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Salma	Sassi	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

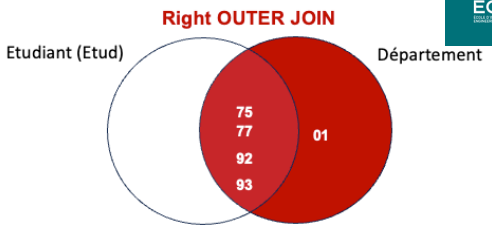
Right OUTER JOIN



Retourner tous les enregistrements de la table de droite même s'il n'y a pas de correspondance avec la table de gauche

Inclut toutes les lignes de la table Département (droite), même celles sans correspondance dans Etudiant (gauche), avec des valeurs NULL.

Jointure externe **(Right OUTER JOIN)**




```
SELECT * FROM Etudiant
RIGHT OUTER JOIN Département  ON dep=dep_num ;
```

Etudiant

Département

nom	prenom	dateN	note	dep	licence	dep_num	dep_nom
Martin	Véra	85-10-31	13.5	77	2	77	Seine-et-Marne
Martin	Annie	85-12-31	15.5	75	1	75	Paris
Dupont	Sylvie	83-02-03	15.0	77	2	77	Seine-et-Marne
Martin	Annie	83-10-22	05.7	93	1	93	Seine-Saint-Denis
Dupond	Laurent	null	null	92	2	92	Hauts-de-Seine
null	null	null	null	null	null	01	Ain

Les colonnes nom, prénom, dateN, note , dep et Licence sont remplies avec des NULL parce qu'il n'y a pas de correspondance avec étudiant.

Jointure externe **(Right OUTER JOIN)**

- **Définition**

  - Une **jointure par la droite** où la table principale (table de droite) est utilisée comme référence.
  - Inclut **toutes les lignes de la table de droite**, même celles **sans correspondance** dans la table de gauche.
  - Les valeurs des colonnes issues de la table de gauche apparaissent comme **NULL** lorsqu'il n'y a pas de correspondance.
  - **Alias :** C'est un **RIGHT OUTER JOIN** par défaut dans SQL.

- **Syntaxe**

```
SELECT * FROM main_table
RIGHT JOIN joined_table
ON main_table.id = joined_table.main_id;
```

- **Cas d'usage**

  1. Obtenir la liste des **départements** avec ou sans **étudiants**.

Jointure externe **(Full OUTER JOIN)**

Inclut toutes les lignes de la table de droite et de la table de gauche

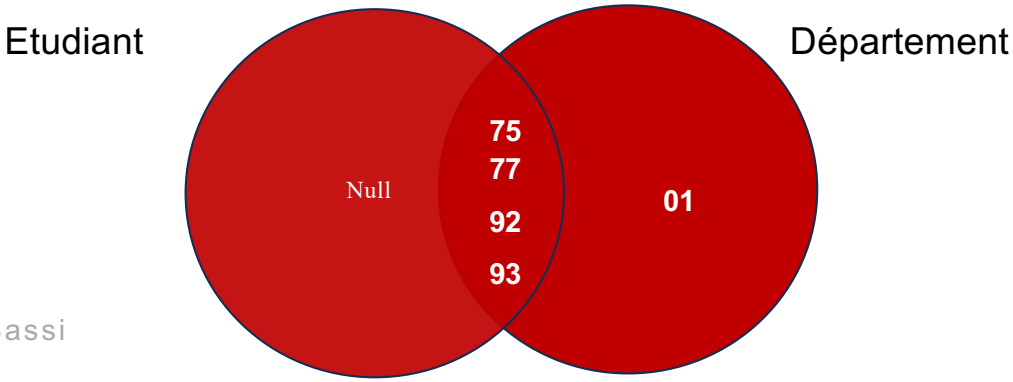
Table 1: Département

dep_num	dep_nom
01	Ain
75	Paris
77	Seine-et-Marne
92	Hauts-de-Seine
93	Seine-Saint-Denis

Table 2: Etudiant

nom	prenom	date_n	note	dep	licence
Martin	Véra	85-10-31	13.5	77	2
Martin	Annie	85-12-31	15.5	75	1
Dupont	Sylvie	83-02-03	15.0	77	2
Martin	Annie	83-10-22	05.7	93	1
Dupond	Laurent			92	2
Lefèvre	Laurent		11.5		3

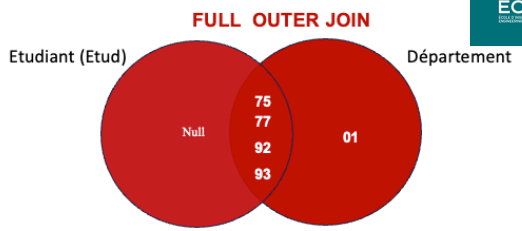
FULL OUTER JOIN



Retourne les lignes communes entre deux tables ainsi que toutes les lignes des deux tables qui n'ont pas de correspondance

Inclut toutes les lignes de la table département (droite) ainsi que celles de la table Etudiant, même s'il n'y a pas de correspondance

Jointure externe **(FULL OUTER JOIN)**



En SQL:
SELECT * FROM Etudiant
FULL OUTER JOIN Département **ON** dep=dep_num ;

Etudiant

Département

nom	prenom	dateN	note	dep	licence	dep_num	dep_nom
Martin	Véra	85-10-31	13.5	77	2	77	Seine-et-Marne
Martin	Annie	85-12-31	15.5	75	1	75	Paris
Dupont	Sylvie	83-02-03	15.0	77	2	77	Seine-et-Marne
Salma	Saassi	83-10-22	05.7	93	1	93	Seine-Saint-Denis
Dupond	Laurent	null	null	92	2	92	Hauts-de-Seine
Lefèvre	Laurent	null	11.5	null	3	null	null
null	null	null	null	null	null	01	Ain

Jointure externe **(FULL OUTER JOIN)**

- **Support MySQL**
  ❌ Non implémenté directement.
  ✅ Peut être simulé avec une combinaison de LEFT JOIN et RIGHT JOIN à l'aide de UNION

- **Syntaxe**

```
SELECT * FROM table1 LEFT JOIN table2 ON table1.id = table2.id
UNION
SELECT * FROM table1 RIGHT JOIN table2 ON table1.id = table2.id;
```

Jointure externe **(FULL OUTER JOIN)**

- **Définition**

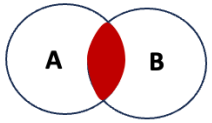
  - Permet de récupérer **toutes les entrées des deux tables**, même si elles n'ont pas de correspondance.
  - **UNION** des résultats de LEFT OUTER JOIN et RIGHT OUTER JOIN.
  - **Alias :** C'est un **FULL OUTER JOIN** par défaut dans SQL.

- **Syntaxe**

```
SELECT *
FROM main_table FULL OUTER JOIN joined_table
ON main_table.id = joined_table.main_id;
```

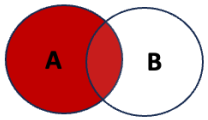
- **Cas d'usage**

  1. Lister tous les **étudiants** et les **départements**, même si un étudiant n'a pas de département et un département n'est affecté à aucun étudiant.



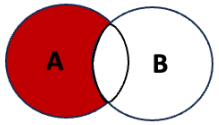

```
SELECT *
FROM A
INNER JOIN B ON A.key = B.key
```

INNER Join : récupère uniquement les enregistrements ayant des correspondances dans les deux tables (A et B).



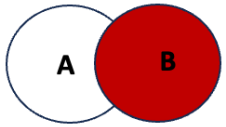

```
SELECT *
FROM A
LEFT JOIN B ON A.key = B.key
```

LEFT JOIN: récupère tous les enregistrements de la table de gauche (A) et les enregistrements correspondants de la table de droite (B).



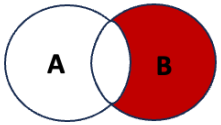

```
SELECT *
FROM A
LEFT JOIN B ON A.key = B.key
WHERE B.key IS NULL
```

LEFT JOIN avec vérification du NULL: filtre uniquement les enregistrements de la table de gauche (A) qui n'ont aucune correspondance dans la table de droite (B).



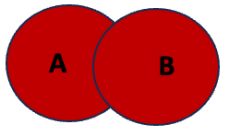

```
SELECT *
FROM A
RIGHT JOIN B ON A.key = B.key
```

RIGHT JOIN: récupère tous les enregistrements de la table de droite (B) et les enregistrements correspondants de la table de gauche (A).



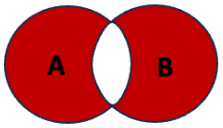

```
SELECT *
FROM A
RIGHT JOIN B ON A.key = B.key
WHERE A.key IS NULL
```

RIGHT JOIN avec vérification du NULL: filtre uniquement les enregistrements de la table de droite (B) qui n'ont aucune correspondance dans la table de gauche (A).




```
SELECT *
FROM A
FULL OUTER JOIN B ON A.key =
B.key
```

FULL OUTER: récupère tous les enregistrements ayant des correspondances ou non dans les deux tables.




```
SELECT *
FROM A FULL OUTER JOIN B ON
A.key = B.key WHERE A.key IS
NULL OR B.key is NULL
```

FULL OUTER avec vérification du NULL: filtre uniquement les enregistrements où il n'y a pas de correspondance dans l'une ou l'autre des tables (A ou B).

Activité

Clubs & Adhérents (Vrai ou Faux)

Table 1: Club

club_id	club_nom
1	Robotique
2	Théâtre
3	Musique
4	Échecs
5	Photo

Table 2: Adhérent

nom	prenom	club_id
Dupont	Alice	1
Martin	Noah	2
Dia	Sana	1
Lefevre	Yann	4
Petit	Lina	NULL
Nguyen	Hugo	NULL

1. **INNER JOIN** affiche uniquement les lignes qui correspondent dans les deux tables. → **Vrai**
2. **LEFT JOIN** peut produire des valeurs NULL dans les colonnes de la table de droite. → **Vrai**
3. **RIGHT JOIN** affiche tous les enregistrements de la table de gauche, même sans correspondance. → **Faux**
4. **FULL OUTER JOIN** combine les résultats de LEFT JOIN et RIGHT JOIN. → **Vrai**
5. Si toutes les valeurs de jointure existent dans les deux tables, **INNER JOIN, LEFT JOIN, RIGHT JOIN et FULL JOIN** donnent le même résultat. → **Vrai**
6. Un **LEFT JOIN** et un **RIGHT JOIN** donnent toujours le même résultat, quel que soit l'ordre des tables. → **Faux**
7. Un **FULL OUTER JOIN** peut produire des lignes avec des NULL des deux côtés. → **Vrai**
8. Le mot-clé **JOIN** sans précision est équivalent à **INNER JOIN**. → **Vrai**
9. On peut chaîner plusieurs jointures dans une seule requête SQL → **Vrai**

Table de gauche: Départ
Table de droite: Adherent

Clubs & Adhérents (Requêtes)

Table 1: Club

club_id	club_nom
1	Robotique
2	Théâtre
3	Musique
4	Échecs
5	Photo

Table 2: Adhérent

nom	prenom	club_id
Dupont	Alice	1
Martin	Noah	2
Dia	Sana	1
Lefevre	Yann	4
Petit	Lina	NULL
Nguyen	Hugo	NULL

Q1. Afficher le nom du club et les adhérents

```
SELECT c.club_nom, a.nom, a.prenom
FROM Club c
INNER JOIN Adhérent a
      ON c.club_id = a.club_id;
```

club_nom	nom	prenom
Robotique	Dupont	Alice
Théâtre	Martin	Noah
Robotique	Dia	Sana
Échecs	Lefevre	Mehdi

Table de gauche: Club
Table de droite: Adhérent

Clubs & Adhérents (Requêtes)

Table 1: Club

club_id	club_nom
1	Robotique
2	Théâtre
3	Musique
4	Échecs
5	Photo

Table 2: Adhérent

nom	prenom	club_id
Dupont	Alice	1
Martin	Noah	2
Dia	Sana	1
Lefevre	Yann	4
Petit	Lina	NULL
Nguyen	Hugo	NULL

Q2. Afficher le nom des clubs avec leurs adhérents. Inclure aussi les clubs qui n'ont aucun adhérent

```
SELECT c.club_nom, a.nom, a.prenom
FROM Club c
LEFT JOIN Adherent a
      ON c.club_id = a.club_id;
```

Club_nom	nom	prenom
Robotique	Dupont	Alice
Robotique	Dia	Sana
Théâtre	Martin	Noah
Musique	NULL	NULL
Échecs	Lefevre	Mehdi
Photo	NULL	NULL

Table de gauche: Club
Table de droite: Adhérent

Clubs & Adhérents (Requêtes)

Table 1: Club

club_id	club_nom
1	Robotique
2	Théâtre
3	Musique
4	Échecs
5	Photo

Table 2: Adhérent

nom	prenom	club_id
Dupont	Alice	1
Martin	Noah	2
Dia	Sana	1
Lefevre	Yann	4
Petit	Lina	NULL
Nguyen	Hugo	NULL

Q3. Afficher tous les adhérents et les infos de leurs clubs.
Afficher même ceux ne sont inscrits dans aucun club.

```
SELECT c.club_nom, a.nom, a.prenom
FROM Club c
RIGHT JOIN Adherent a
      ON c.club_id = a.club_id;
```

club_nom	nom	prenom
Robotique	Dupont	Alice
Robotique	Dia	Sana
Théâtre	Martin	Noah
Échecs	Lefevre	Mehdi
NULL	Petit	Lina
NULL	Nguyen	Hugo

Table de gauche: Club
Table de droite: Adhérent

Clubs & Adhérents (Requêtes)

Table 1: Club

club_id	club_nom
1	Robotique
2	Théâtre
3	Musique
4	Échecs
5	Photo

Table 2: Adhérent

nom	prenom	club_id
Dupont	Alice	1
Martin	Noah	2
Dia	Sana	1
Lefevre	Yann	4
Petit	Lina	NULL
Nguyen	Hugo	NULL

Q4. Afficher le nom des clubs et leurs adhérents. Inclure tous les clubs et tous les adhérents, même sans correspondance.

```
SELECT c.club_nom, a.nom, a.prenom
FROM Club c
FULL OUTER JOIN Adherent a
      ON c.club_id = a.club_id;
```

club_nom	nom	prenom
Robotique	Dupont	Alice
Robotique	Dia	Sana
Théâtre	Martin	Noah
Musique	NULL	NULL
Échecs	Lefevre	Mehdi
Photo	NULL	NULL
NULL	Petit	Lina
NULL	Nguyen	Hugo

Les sous-requêtes

- Une **sous-requête** est une requête SQL placée à l'intérieur d'une autre requête.
- Lorsqu'elle contient une **fonction d'agrégation** (AVG, SUM, COUNT, MIN, MAX), on parle d'**agrégation dans une sous-requête**.

- Le résultat de cette requête (souvent une seule valeur) peut être utilisé :
  - dans un **WHERE** pour filtrer,
  - dans un **HAVING** pour restreindre des groupes,
  - ou dans un **SELECT** pour enrichir le résultat.

Les sous-requêtes

Exemple

Trouver les étudiants qui ont une note > à la **moyenne générale** :

- SELECT nom, prenom, note FROM etudiants
  WHERE note > (SELECT AVG (note) FROM etudiants);

Nom	Prénom	note
Martin	Véra	13.5
Martin	Annie	15.5
Dupont	Sylvie	15.0

Les vues

- Une **vue** est une **table virtuelle** définie par une requête.
- Elle est particulièrement utile pour **conserver et réutiliser** des résultats **agrégés**.
- Cela évite de réécrire sans cesse les mêmes requêtes avec GROUP BY, HAVING, etc.

👉 Dans l'analyse de données, la vue sert de **couche intermédiaire** :
  - On agrège une fois,
  - Puis on interroge la vue comme une table.

Les vues

Exemple

Créer une table virtuelle (vue) qui contient, pour chaque département, la moyenne des notes des étudiants.

```
CREATE VIEW moyennes_par_dep AS
SELECT dep, AVG(note) AS moyenne_note FROM etudiants GROUP BY dep;
```

dep	moyenne_note
01	11.5
75	15.5
77	14.25
92	null
93	5.7

Etude de cas
Gestion d'une bibliothèque

Contexte
La bibliothèque municipale souhaite mieux gérer ses livres et ses emprunts.

Elle dispose des données suivantes :

- **Livres** : identifiant, titre, auteur, année de publication, id_categorie.
- **Catégories** : identifiant, nom de la catégorie.
- **Adhérents** : identifiant, nom, prénom, ville, âge.
- **Emprunts** : relie les adhérents aux livres (date d'emprunt, date de retour).

Etude de cas
Gestion des étudiants et départements

Table Livre

id_livre	titre	auteur	annee	id_categorie
1	Le Petit Prince	A. de Saint-Exupéry	1943	10
2	L'Étranger	A. Camus	1942	11
3	Les Misérables	V. Hugo	1862	10
4	Fondations	I. Asimov	1951	12
5	Harry Potter	J.K. Rowling	1997	10

Etude de cas
Gestion des étudiants et départements

Table Catégorie

d_categorie	nom_categorie
10	Roman
11	Philosophie
12	Science-Fiction

Etude de cas
Gestion des étudiants et départements

Table Adhérent

id_adherent	nom	prenom	ville	age
101	Martin	Alice	Lyon	25
102	Dupont	Karim	Pau	32
103	Bernard	Eva	Bayonne	41
104	Simon	Paul	Lyon	19
105	Leroy	Clara	Pau	28

Etude de cas
Gestion des étudiants et départements

Table Emprunt

id_emprunt	id_adherent	id_livre	date_emprunt	date_retour
1001	101	1	2025-01-10	2025-01-25
1002	102	2	2025-01-12	NULL
1003	103	3	2025-01-15	2025-01-30
1004	104	4	2025-01-20	NULL
1005	105	5	2025-01-22	2025-02-05

Etude de cas

Gestion des étudiants et départements

Travail demandé

Rédigez les requêtes SQL suivantes :

1. Nombre total de livres dans la bibliothèque.
2. Nombre d'adhérents par ville.
3. Moyenne d'âge des adhérents.
4. Nombre de livres par catégorie.
5. Liste des adhérents ayant emprunté plus d'un livre
6. Liste des emprunts non encore rendus (date_retour = NULL).
7. Livre le plus ancien et le plus récent (MIN / MAX).
8. Créer une **vue** Vue_StatCategorie qui affiche :
   nom_categorie, nb_livres, nb_emprunts.
9. Adhérents dont l'âge est supérieur à la moyenne générale (sous-requête).

ECE
ÉCOLE D'INGÉNIEURS
ENGINEERING SCHOOL

Au revoir

Dr. Salma Sassi
salma.sassi@ece.fr